

DP Computer Science: A 3.3.4 SQL Aggregate Functions (HL Only)

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of this lesson, students will be able to:

- Identify and describe the purpose of the **five main SQL aggregate functions**: `AVG` , `COUNT` , `MAX` , `MIN` , and `SUM`
- Construct SQL `SELECT` statements that utilise these aggregate functions
- Apply the `GROUP BY` clause to categorise data and perform calculations on these groups
- Write SQL queries that use aggregate functions with the `GROUP BY` clause to extract meaningful information
- Interpret the results of SQL queries involving aggregate functions and grouped data

Key Vocabulary

Term	Definition
Aggregate function	A function that performs a calculation on a set of values and returns a single value
AVG	Calculates the average (mean) of a numeric column
COUNT	Counts the number of rows in a result set
MAX	Returns the maximum value in a column
MIN	Returns the minimum value in a column
SUM	Calculates the total sum of a numeric column
GROUP BY	Groups rows that have the same values in specified columns
HAVING	Filters groups after aggregation (like WHERE for groups)
NULL	A missing or unknown value; aggregate functions usually ignore NULLs

Relevant ATL Skills

Skill	Description
Thinking	Analysis and evaluation; problem-solving; applying concepts
Communication	Interpreting database tables and query results; presenting information logically
Self-Management	Organising SQL code; managing time during practical exercises

2. Introduction to Aggregate Functions

What are Aggregate Functions?

Aggregate functions perform calculations on a set of values and return a single value. They are used for data analysis, reporting, and decision-making.

The Five Main Aggregate Functions

text

AGGREGATE FUNCTIONS

AVG

Calculates the average value
(Mean of numeric values)

COUNT

Counts the number of rows

MAX

Returns the maximum value

MIN

Returns the minimum value

SUM

Calculates the total sum

Aggregate Function Syntax

sql

```
SELECT aggregate_function(column_name)
FROM table_name
WHERE condition;
```

3. Detailed Explanation of Each Function

AVG (Average)

Calculates the average (mean) value of a numeric column.

Aspect	Description
Purpose	Find the average of a set of values

Aspect	Description
Returns	Numeric value (average)
NULL Handling	Ignores NULL values
Example	<code>SELECT AVG(Salary) FROM Employees;</code>

```
sql

-- Find the average salary of all employees
SELECT AVG(Salary) AS AverageSalary
FROM Employees;

-- Find the average salary of employees in a specific department
SELECT AVG(Salary) AS AverageSalary
FROM Employees
WHERE Department = 'IT';
```

COUNT

Counts the number of rows in a result set.

Aspect	Description
Purpose	Count total rows or non-NULL values
Returns	Integer (count)
NULL Handling	<code>COUNT(*)</code> counts all rows; <code>COUNT(column)</code> counts non-NULL values
Example	<code>SELECT COUNT(*) FROM Employees;</code>

```
sql

-- Count all employees
SELECT COUNT(*) AS TotalEmployees
FROM Employees;

-- Count employees with a non-NULL salary
SELECT COUNT(Salary) AS EmployeesWithSalary
FROM Employees;

-- Count employees in the IT department
SELECT COUNT(*) AS ITEmployees
FROM Employees
WHERE Department = 'IT';
```

MAX (Maximum)

Returns the maximum value in a column.

Aspect	Description
Purpose	Find the highest value
Returns	Same data type as the column

Aspect	Description
NULL Handling	Ignores NULL values
Example	<code>SELECT MAX(Salary) FROM Employees;</code>

```
sql

-- Find the highest salary
SELECT MAX(Salary) AS HighestSalary
FROM Employees;

-- Find the most recent hire date
SELECT MAX(HireDate) AS MostRecentHire
FROM Employees;

-- Find the highest salary in the IT department
SELECT MAX(Salary) AS HighestITSalary
FROM Employees
WHERE Department = 'IT';
```

MIN (Minimum)

Returns the minimum value in a column.

Aspect	Description
Purpose	Find the lowest value
Returns	Same data type as the column
NULL Handling	Ignores NULL values
Example	<code>SELECT MIN(Salary) FROM Employees;</code>

```
sql

-- Find the Lowest salary
SELECT MIN(Salary) AS LowestSalary
FROM Employees;

-- Find the earliest hire date
SELECT MIN(HireDate) AS EarliestHire
FROM Employees;

-- Find the Lowest salary in the IT department
SELECT MIN(Salary) AS LowestITSalary
FROM Employees
WHERE Department = 'IT';
```

SUM (Sum)

Calculates the total sum of a numeric column.

Aspect	Description
Purpose	Add up all values in a column

Aspect	Description
Returns	Numeric value (total)
NULL Handling	Ignores NULL values
Example	<code>SELECT SUM(Salary) FROM Employees;</code>

```
sql

-- Calculate total salary paid to all employees
SELECT SUM(Salary) AS TotalSalary
FROM Employees;

-- Calculate total salary in the IT department
SELECT SUM(Salary) AS TotalITSalary
FROM Employees
WHERE Department = 'IT';
```

4. Using Multiple Aggregate Functions

Example: All Functions in One Query

```
sql

SELECT
    COUNT(*) AS TotalEmployees,
    AVG(Salary) AS AverageSalary,
    MAX(Salary) AS HighestSalary,
    MIN(Salary) AS LowestSalary,
    SUM(Salary) AS TotalSalary
FROM Employees;
```

Example Output

TotalEmployees	AverageSalary	HighestSalary	LowestSalary	TotalSalary
25	52,000	95,000	30,000	1,300,000

5. The GROUP BY Clause

Why Use GROUP BY?

Aggregate functions without `GROUP BY` calculate a single value for the entire table. `GROUP BY` allows you to calculate separate aggregate values for each group (category).

Basic Syntax

```
sql

SELECT column1, aggregate_function(column2)
FROM table_name
GROUP BY column1;
```

Example with GROUP BY

```
sql
-- Calculate the average salary for each department
SELECT
    Department,
    AVG(Salary) AS AverageSalary
FROM Employees
GROUP BY Department;
```

Example Output

Department	AverageSalary
IT	65,000
HR	45,000
Sales	55,000
Finance	60,000

Visual Representation

text

GROUP BY EXPLANATION

Without GROUP BY:

SELECT AVG(Salary) FROM Employees;

Returns ONE value: 52,000

With GROUP BY:

SELECT Department, AVG(Salary)

FROM Employees

GROUP BY Department;

Department	AverageSalary
IT	65,000
HR	45,000
Sales	55,000

Important Rule: GROUP BY Columns

Any non-aggregated column in the `SELECT` statement must appear in the `GROUP BY` clause.

```
sql
--  Correct: Department is in GROUP BY
SELECT Department, AVG(Salary)
FROM Employees
```

```
GROUP BY Department;

-- ❌ Incorrect: FirstName is not in GROUP BY
SELECT Department, FirstName, AVG(Salary)
FROM Employees
GROUP BY Department;
```

6. GROUP BY with Multiple Columns

Example: Grouping by Two Columns

```
sql

-- Calculate average salary by department and job title
SELECT
    Department,
    JobTitle,
    AVG(Salary) AS AverageSalary
FROM Employees
GROUP BY Department, JobTitle;
```

Example Output

Department	JobTitle	AverageSalary
IT	Developer	68,000
IT	Manager	85,000
HR	Assistant	40,000
HR	Manager	55,000

7. Additional Considerations

Dealing with NULL Values

Function	NULL Handling
AVG	Ignores NULL values
COUNT(*)	Counts all rows (including NULLs)
COUNT(column)	Counts non-NULL values
MAX	Ignores NULL values
MIN	Ignores NULL values
SUM	Ignores NULL values

Aliases

Use aliases (`AS`) to make query results more readable:

```
sql

SELECT
    Department,
    COUNT(*) AS EmployeeCount,
    AVG(Salary) AS AverageSalary
FROM Employees
GROUP BY Department;
```

DISTINCT with COUNT

```
sql

-- Count distinct departments
SELECT COUNT(DISTINCT Department) AS DepartmentCount
FROM Employees;
```

8. Common Mistakes and Best Practices

Common Mistakes

Mistake	Example	Correction
Missing GROUP BY column	<code>SELECT Department, FirstName, AVG(Salary)</code>	Add FirstName to GROUP BY
Wrong column in GROUP BY	<code>GROUP BY DepartmentID</code> instead of <code>Department</code>	Use correct column
Misplaced WHERE/HAVING	Using HAVING for row filtering	Use WHERE for rows; HAVING for groups

Best Practices

Practice	Description
Use aliases	Make results readable with <code>AS</code>
Test with small data	Test queries on small datasets first
Check for NULLs	Consider how NULL values affect results
Document queries	Add comments for complex queries

9. The HAVING Clause (Optional Introduction)

What is HAVING?

`HAVING` filters groups after aggregation. It is like `WHERE` but for groups.

Syntax


```
sql

SELECT Department, AVG(Salary) AS AverageSalary
FROM Employees
GROUP BY Department
HAVING AVG(Salary) > 50000;
```

WHERE vs. HAVING

Clause	When Applied	Purpose
WHERE	Before grouping	Filters individual rows
HAVING	After grouping	Filters groups

Example: HAVING in Action

```
sql

-- Find departments with an average salary over 50,000
SELECT Department, AVG(Salary) AS AverageSalary
FROM Employees
WHERE HireDate > '2020-01-01'
GROUP BY Department
HAVING AVG(Salary) > 50000;
```

10. Practical Examples

Example 1: Student Database

Schema: Students(StudentID, FirstName, LastName, GradeLevel, GPA, Subject)

```
sql

-- Count students by grade level
SELECT GradeLevel, COUNT(*) AS StudentCount
FROM Students
GROUP BY GradeLevel
ORDER BY GradeLevel;
```

```
sql

-- Calculate average GPA by subject
SELECT Subject, AVG(GPA) AS AverageGPA
FROM Students
GROUP BY Subject;
```

```
sql

-- Find the highest and lowest GPA
SELECT MAX(GPA) AS HighestGPA, MIN(GPA) AS LowestGPA
FROM Students;
```

Example 2: Orders Database

Schema: Orders(OrderID, CustomerID, OrderDate, Total, Status)

```
sql

-- Total sales by status
SELECT Status, SUM(Total) AS TotalSales
FROM Orders
GROUP BY Status;
```

```
sql

-- Average order value by customer
SELECT CustomerID, AVG(Total) AS AverageOrderValue
FROM Orders
GROUP BY CustomerID
ORDER BY AverageOrderValue DESC;
```

11. Lesson Sequence

Lesson Plan (60 minutes)

Phase	Time	Activity
Introduction	5 min	Welcome; recap of previous SQL concepts; lesson objectives
Introducing Aggregate Functions	10 min	Definition; introduction to AVG and COUNT
More Aggregate Functions	10 min	Explanation of MAX, MIN, SUM with syntax and examples
GROUP BY Clause	10 min	Introduction; syntax; examples; important rules
HAVING (Optional)	5 min	Brief introduction to HAVING for filtering groups
Practical Activity	15 min	Students work through worksheet using DB Browser for SQLite
Summary	5 min	Review key concepts; Q&A

Practical Activity: Worksheet Tasks

Task	Description
Task 1	Count the total number of students in the database
Task 2	Calculate the average GPA of all students
Task 3	Find the maximum and minimum GPA
Task 4	Count students by grade level (GROUP BY)

Task	Description
Task 5	Calculate average GPA by subject (GROUP BY)
Task 6	Find total sales by department (GROUP BY)

12. Assessment Activities

Assessment Type	Description
Class Participation	Observe engagement and contributions
Worksheet	Evaluate ability to use aggregate functions and GROUP BY
Exit Ticket	Gauge understanding and identify areas needing clarification

Sample Exit Ticket Questions:

1. What does the AVG function do?
2. What is the purpose of the GROUP BY clause?
3. Write a query to count the number of students in each grade level.
4. Write a query to calculate the average salary in each department.

13. Differentiation

Level	Strategy
Support	Provide pre-written query structures with blanks; pair with more confident students
Challenge	Use HAVING clause; combine multiple aggregate functions; work with complex schemas

14. Resources

- **Presentation** (Slide Deck)
- **eBook** (A3.3.4 with examples)
- **Worksheet:** Aggregate functions and GROUP BY exercises
- **Worksheet Answers**
- **DB Browser for SQLite**

15. Summary: Key Takeaways

Function	Purpose	Example
AVG	Average of values	AVG(Salary)
COUNT	Count rows	COUNT(*)
MAX	Maximum value	MAX(Salary)

Function	Purpose	Example
MIN	Minimum value	MIN(Salary)
SUM	Total sum	SUM(Salary)
GROUP BY	Groups rows for aggregation	GROUP BY Department

Key Rules

text

KEY RULES

Aggregate functions calculate a single value

GROUP BY divides data into groups

Non-aggregated columns in SELECT must be in GROUP BY

WHERE filters rows before grouping

HAVING filters groups after aggregation

Aggregate functions ignore NULL values

"Aggregate functions transform raw data into meaningful insights—they are the foundation of data analysis in SQL."